

Traffic

Digital Twin

Agent-executable Product
Requirements Document

ITD video analytics | Concurrent processing
Lightweight forecasting | Virtual signal decisions

VERSION	1.0.1
PREPARED	22 September 2026
SCOPE	12 video assets; C1-C6 aggregate demonstration
BASELINE	main / d6d774f6fb187ffc07925e0768add15ab53d548c
STATUS	Specification only; implementation evidence pending

Existing videos -> ITD + ByteTrack -> aggregate observations
-> video-derived boundary demand -> conserved traffic model
-> causal forecasts -> operator-approved virtual timing

No SUMO. No physical signal control. No invented measurements.
Agent-led build; evidence-based readiness.

ITD v1.2 status: Already downloaded (owner-confirmed); local checkpoint verification pending.

Review boundary: static source and documentation. No application tests, complete 12-video processing or checkpoint execution were performed while preparing this PRD. No repository changes were pushed.

Document map

Lead agent: read Sections 1-8, freeze the contracts in Section 12, then execute the task DAG in Section 23. The final section and appendices define sources, setup and change control.

- [1. Reading guide, authority and evidence](#)
- [2. Consolidated decisions from the conversation](#)
- [3. Product problem, users and success](#)
- [4. Scope and non-goals](#)
- [5. Repository baseline and required corrections](#)
- [6. Target architecture and modes](#)
- [7. Agent autonomy, permissions and evidence](#)
- [8. What to obtain, where and how](#)
- [9. Media inventory and camera mapping](#)
- [10. Automated geometry and data quality](#)
- [11. Detection, tracking and measurements](#)
- [12. Time semantics and authoritative data contracts](#)
- [13. Video-derived demand adapter and conservation](#)
- [14. Aggregate engine requirements](#)
- [15. Lightweight causal forecasting](#)
- [16. Signal recommendations and operational safety](#)
- [17. Parallel runtime and scheduling](#)
- [18. Auto-annotation, agent review and fine-tuning policy](#)
- [19. APIs, persistence and UI](#)
- [20. Privacy, security and permitted use](#)
- [21. Validation plan and acceptance gates](#)
- [22. File-by-file implementation map](#)
- [23. Execution workstreams, dependencies and effort](#)
- [24. Agent command and artifact contract](#)
- [25. Demo rehearsal and rollback](#)
- [26. Commercial, scale and cost decisions](#)
- [27. Risk register and unresolved inputs](#)
- [28. Requirement traceability and definition of done](#)
- [Appendix A. Downloaded-video source register from the conversation](#)
- [Appendix B. Source register](#)
- [Appendix C. Installation and verification playbook](#)
- [Appendix D. Delivery notes and change control](#)

Source notation: C = conversation; R = pinned repository; E = official documentation. New defaults, schemas and acceptance targets are proposed design requirements, not measured performance.

1. Reading guide, authority and evidence

1.1 Is PRD ko kaise use karna hai

Lead coding agent sabse pehle Sections 1-8 padhe, phir contracts / time semantics freeze kare, aur Section 23 ka dependency graph execute kare. Vision, model aur UI agents apne workstreams parallel chala sakte hain, lekin shared files ka single owner hoga. QA agent acceptance criteria implement karega; test names ya status prose completion evidence nahi hain.

Companion AGENT_START_HERE.md, agent_tasks.json, source_registry.json aur configuration templates isi PRD ke execution aids hain. Canonical requirement content yeh Markdown file hai. Word/PDF reading copies hain; changes canonical file aur version log mein reflect karna mandatory hai.

1.2 Evidence labels

Label	Meaning	Use rule
C	User conversation / uploaded migration report	Requirement or historical context; benchmark proof nahi.
R	Pinned repository source / branch verification	Source behavior or configured value; runtime success ka proof nahi.
E	Official external documentation / primary research	Tool capability / algorithm reference; project accuracy ka proof nahi.
D	New design decision in this PRD	Implementation requirement or proposed acceptance target, not measured result.

All prescriptive MUST/SHALL statements, new schemas, defaults, thresholds and task definitions are **D: proposed requirements** unless explicitly attributed to a source. Numeric examples are illustrative. Agent must not turn a planning target into a reported measurement.

1.3 Latest intent ko preserve karna

User ne pehle synchronized Indian CCTV network maanga tha; ab 12 downloaded, potentially unrelated sample videos demo ke liye accept ki hain. User ne later concurrent processing aur agent-led implementation require kiya. Isliye earlier suggestions ko silently overwrite nahi kiya gaya: decisions aur their trade-offs Section 2 mein explicit hain. Unknown model filename, hardware, paths, permissions aur date agent discovery outputs rahenge; yahan guesses nahi hain. [C01]

2. Consolidated decisions from the conversation

Topic	Earlier discussion	Final requirement / explicit resolution
Data	Same-city, synchronized, connected CCTV	Current demo uses existing 12 clips. No fresh dataset hunt on the critical path. Real connectivity/synchronization not claimed.
Simulator	SUMO-based runtime	Preserve SUMO removal. Repair finite-capacity aggregate engine instead of reintroducing per-vehicle simulation.
Topology	Three signals / six cameras in an early sketch	Retain actual C1-C6: two controlled nodes, four boundaries. Six operational observation slots; twelve media assets/sessions are separate concepts.
Detector	BMD/ITD training discussed	ITD v1.2 is already downloaded (owner-confirmed). Reuse and verify the local checkpoint; no default redownload or training prerequisite.

Topic	Earlier discussion	Final requirement / explicit resolution
Tracker	Header mentioned ByteTrack	Actual ByteTrack implementation required; old IoU/distance heuristic is not accepted as ByteTrack.
Forecast	Persistence, then possible large ML models	Persistence baseline + EWMA default after warm-up; damped Holt is an optional evaluated challenger. No LLM numerical controller.
Cached vs online	Precompute-first recommended for presentation reliability	Both cached playback and concurrent online inference on recorded clips are required modes. Cache is not evidence of real-time throughput.
Manual work	Human geometry / annotation review suggested	Agent does all routine setup, coding, draft geometry, annotation and review preparation. Autonomous provisional demo operation allowed; missing independent validation is disclosed, never fabricated.
Signal authority	Operator approval / manual takeover	Retain authority. Agent may run scripted test-operator actions in isolated tests, not impersonate a real approving operator in an operational run.
Production	Scaling discussed	Production is a roadmap, not this release's completion label. Commercial rights, real calibration and field validation are separate gates.

Not silently resolved: agent-only labels cannot become independently established ground truth. Unrelated MP4s cannot become synchronized real corridor evidence. Unlimited automation cannot create missing account permissions, legal rights, GPU capacity or unseen local files. These are capability/authorization gaps, not tasks to fake. [C01]

3. Product problem, users and success

3.1 Problem statement

Existing code mein synthetic traffic engine / forecasts aur sample-video analytics disconnected hain. User ko aisa MVP chahiye jahan video se directional traffic measurement visibly nikle, connected synthetic roads mein traffic propagate ho, future congestion risk explain ho, aur approved virtual signal plan ka modeled consequence dikhe. Individual real-vehicle identities ya physical roadside actuation required nahi. [C01][R03][R04]

3.2 Primary users

Founder/demo operator scenario, cameras, forecasts, alternative plans aur audit inspect karega. Coding agents implementation / evidence produce karenge. Reviewer engineering correctness, provenance aur limitations verify karega. Future municipal users are not current live-control users; no government deployment approval is implied.

3.3 End-to-end success story

Operator ek usable clip select karta hai. UI actual MP4 dikhati hai; detector/tracker se generated, timestamp-aligned directional counts appear hote hain. Four declared boundary inputs selected video profiles se supply hote hain. C1/C3 virtual lights road discharge change karte hain. Forecast worker frozen state se future outcomes calculate karta hai. Operator baseline/candidate compare karke approve, modify ya reject karta hai. Approval correct phase boundary par apply hota hai. Camera video unchanged rehti hai; modeled graph changes. Audit, missing-data behavior aur replay independently inspectable hote hain.

3.4 Separate success labels

ENGINEERING_DEMO_READY: functional integration, deterministic replay, source labels and invariant tests passed.
CONCURRENT_INFERENCE_SUPPORTED: multiple independent camera sessions actually executed. REALTIME_12_

VALIDATED: twelve-camera target-machine benchmark separately passed. ACCURACY_VALIDATED: reviewed independent reference set and specified metrics passed. COMMERCIAL_RIGHTS_CLEAR: documented rights for intended use. FIELD_READY: outside this PRD.

One label doosre ko imply nahi karta. Agent-only review ke saath engineering demo release ho sakta hai, but accuracy/field/rights badges unresolved reh sakte hain.

4. Scope and non-goals

4.1 P0: required implementation

P0 includes 12-asset inventory, ITD smoke test, actual ByteTrack, per-camera geometry, bidirectional counting where visible, visible queue estimates where supported, quality states, JSONL observations, source-time synchronization, cached playback, concurrent recorded-video inference, four-boundary demand adapter, aggregate-engine correctness repairs, causal EWMA/persistence forecasts, bounded candidate comparisons, actual MP4 UI, existing operator workflows, audit, isolated tests and a reproducible demo package.

P0 must implement all twelve session slots without changing graph topology. A source that is unreadable/moving/unsuitable can be rejected with evidence; the inventory must still account for it. Twelve simultaneous successful inferences require twelve usable sources and adequate compute; an unavailable source cannot be replaced with an undisclosed duplicate.

4.2 Conditional / optional work

Auto-annotation export and agent review reports are required automation capabilities. CVAT UI/server installation is optional unless review needs it; local interoperable annotation export must work without CVAT. Fine-tuning is conditional on detector error evidence, permitted-use rights and approved compute budget. Damped Holt, ONNX Runtime and TensorRT are optional challengers/optimizations after baseline correctness. Future authorized CCTV ingest/state assimilation is a documented extension, not a hidden MVP dependency.

4.3 Do not build

No SUMO/TraCI/netconvert dependency, city-wide vehicle tracking, ANPR, facial recognition, cross-camera re-identification, generic LLM in the numerical runtime, deep neural forecasting from short clips, RL/GNN signal controller, Kafka/Redis/Kubernetes/Triton deployment by default, extra public Python HTTP gateway, separate DB-writer service, auto-purchased cloud GPUs, camera hacking, dataset scraping or physical-signal integration. No UI rewrite or toolchain migration merely because an old README mentions another framework. [C01][R02]

5. Repository baseline and required corrections

5.1 Baseline verification

On preparation, main resolved to d6d774f6fb187ffc07925e0768add15ab53d548c, commit message "Remove legacy external traffic simulator artifacts". Agent must resolve HEAD again before editing; preserve uncommitted user work and record differences. Pinned sources below are evidence references, not instructions to reset the repository backwards. [R01]

Area / source	Source-derived finding	Required treatment
services/vision/pipeline.py	MOG2/contours, size/colour classifier; manual IoU/distance association; fixed regions; per-frame accumulation.	Replace default perception; bound retention; preserve missing-data semantics. [R03]
aggregate_engine.py, demand.py	Aggregate runtime consumes BoundaryDemand.next(tick) and produces link summaries.	Inject demand provider; keep runtime lifecycle/receipts. [R04]

Area / source	Source-derived finding	Required treatment
flow_kernel.py	Finite-capacity flow, shared receiving space; backward-wave parameter not used; output values clamped.	Repair sending/receiving and assert conservation before toleranced cleanup. [R05]
metrics.py	Queue threshold rule, stock-based density and 5% free-speed floor.	Version metrics; remove unsupported floor; distinguish queue / stock. [R06]
intelligence/model.py	Reconstructs cells from summaries; initializes backlogs to zero; evaluates finite plan set.	Complete snapshots; preserve backlogs; retain bounded evaluator. [R07]
forecast_demand.py	Uses admitted inflow as future boundary rate.	Forecast offered boundary demand from causal history, not blocked admission alone. [R08]
vision-analytics-panel.tsx, vision-data.ts	Illustrated canvas, static fallback and outdated track/speed/impact types.	Actual MP4, explicit modes, new typed observations, no silent fallback. [R09]
twin.proto	Aggregate link state present; no general camera-observation / demand-profile command interface.	Add new messages/fields with compatibility tests; regenerate clients. [R10]
requirements.lock	Shared lock lacks actual CV stack imported by vision source.	Isolated vision environment with verified versions. [R11]
Scenario configuration	C1/C3 controlled; C2/C4/C5/C6 boundaries; 10 links and 14 movements.	Keep topology; separate camera mapping. [R12]
Intelligence tests	Some named conservation tests only assert non-negativity.	Add full inventory identity / exact-continuation tests. [R13]

5.2 Correctness backlog before headline forecasts

FIX-01: complete state. Eliminate summary-to-cell reconstruction in authoritative demo prediction. Preserve every cell stock, per-boundary backlog, scheduler stage/remaining time, pending plan, eligible-service history, active disclosed restrictions and versions. Summary reconstruction is an explicitly degraded import path only. [R04][R07]

FIX-02: offered versus admitted demand. Maintain offered-demand history, admitted transfers and external backlog separately. Blocked admission must not be interpreted as vanishing demand. [R08]

FIX-03: metric identity. Forecast storage utilization is full stock / physical storage. Queued stock / storage is a different ratio. Congestion threshold crossing is not automatically physical spillback. Use separate risk types and field names. [R06][R07]

FIX-04: traffic mathematics. Apply configured backward-wave receiving behavior; validate stable numerical step and capacity consistency. Final value clamps must not hide model-created/lost mass. [R05][R12]

FIX-05: junction turns. Under strict FIFO, a positive-share red or blocked turn prevents unsupported shared-approach release. Never subtract total release but add only a subset of turning shares. Test partial-permission configurations even if current phases normally enable a whole approach. This is a required robustness test inferred from the current allocator structure. [R05]

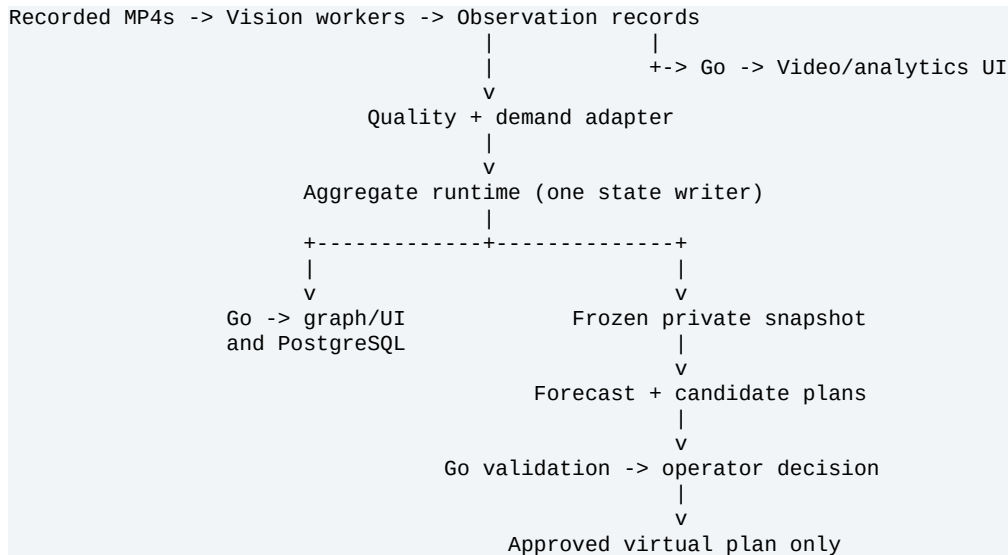
FIX-06: timing, incidents and emergency. Known active capacity restriction can persist as an explicit forecast assumption; unseen scenario end-time is not available to the predictor. Emergency progress is modeled, not GPS. Preserve clearances, operator locks and priority cancellation. No guessed exact ambulance ETA. [C01][R04]

FIX-07: reporting. Queue delay uses vehicle-seconds; boundary exits are throughput; waiting/service debt is not the oldest real vehicle's wait. Model speed must not impose an unexplained positive floor at total blockage. Preserve deprecated fields only for deliberate compatibility, not misleading UI claims. [R06][R07][R10]

6. Target architecture and modes

6.1 Ownership boundaries

Browser communicates with Go only. Go owns authentication, registered asset access, public REST/WebSocket, run ownership, audit, approval and authoritative PostgreSQL writes. Python vision owns decode/inference/tracking/geometry/aggregation. Python aggregate runtime owns physical synthetic state and virtual signals. Python intelligence owns demand estimates and candidate rollouts. Optional annotation tooling is development-only. Build agents are not runtime traffic services. [R02]



6.2 Orthogonal mode fields

processing_mode is cached_observations or online_inference. media_source is recorded_video for these clips, never live_cctv. demand_source is seeded or video_profile. Existing operator_mode remains observe, recommend or manual. control_target is always virtual_only. Separate these fields; one overloaded live Boolean is forbidden.

Mode A: prepared demonstration. Preprocess observations once, play actual videos with saved measurements, and execute traffic dynamics / plan comparisons during the demo. Caption: "Recorded video; precomputed observations; live virtual simulation."

Mode B: concurrent inference demonstration. Decode/process selected recorded clips during the run. Shared inference, isolated trackers and independent forecasting overlap. Caption: "Recorded video; online inference; virtual simulation." Support all twelve registered sessions; benchmark capability separately.

Mode C: seeded reference. Existing reproducible scenarios remain available without ITD, video decoding or a GPU. Missing vision installation must not break this mode.

Mode changes affecting authoritative data end the old run, invalidate outstanding recommendations and start a new run/version. A cached fallback is a visible, user-selected change, not an automatic claim that online inference is still running.

7. Agent autonomy, permissions and evidence

7.1 Agent-first operating requirement

User routine coding, package installation, manual box drawing, file renaming aur test-report writing nahi karega. Lead agent owns discovery, setup, implementation, asset registration, draft geometry, automated annotation, automated checks, review artifacts, benchmark runs and handoff. Specialist agents can review each other's outputs. Where independent human review is not available, the system continues in a clearly labeled engineering/provisional mode rather than pretending human validation happened.

7.2 Permitted, approval-gated and prohibited actions

Action class	Default policy
Workspace source reads; isolated branch edits; local scripts/tests	Agent executes after workspace access is granted.
Approved package downloads / checkpoint reuse	Agent executes within network allowlist and intended-use permissions; record source/hash.
Draft geometry and labels; agent-reviewed demo selection	Agent executes; tag agent_accepted_demo / agent_reviewed, retain evidence.
Account login, license acceptance, paid compute, external media upload	Owner authorization required; agent prepares exact request and cost/data scope.
Driver/OS changes, raw-asset deletion, main-branch push, public release	Explicit authorization required; default deny.
Invented approvals, bypassed license gates, fabricated results, physical signals	Prohibited.

Only genuine non-resolvable prerequisites should interrupt the owner. Bundle missing paths, permissions and budget questions once. Continue independent code/test work when a data-dependent task is blocked. Do not repeatedly ask the user to perform tasks the agent can execute with granted tools.

7.3 Validation provenance

Every geometry/annotation decision stores reviewer_type, reviewer_id, timestamp, evidence hash and validation_level. Allowed levels: unreviewed, agent_reviewed, independent_reference_reviewed. Another model agreeing with ITD is useful diagnostic evidence but not independent physical ground truth. A coding agent cannot sign a nonexistent human approval or classify its own labels as objectively correct.

7.4 Worktrees and coordination

Use one tool initially: local Codex or Antigravity. Both have documented workspace/worktree workflows; capability does not guarantee correctness or hardware access. Use official setup pages and current permission controls. Do not enable unrestricted whole-machine access as the default. [E01][E02]

Lead agent owns contracts, task graph and merges. Vision agent owns services/vision and media metadata. Dynamics agent owns flow/kernel/snapshot/forecast logic. UI agent owns media/analytics views against frozen contracts. QA agent owns independent tests/evidence. Each uses a separate branch/worktree. One machine-level GPU lease prevents parallel training/benchmark runs from invalidating measurements. No background run is assumed to persist after the selected agent environment stops.

8. What to obtain, where and how

8.1 Discovery before installation

Confirmed existing asset: ITD v1.2 pretrained model is already downloaded (owner-confirmed). The agent **MUST** locate and reuse the existing checkpoint; no model download by default. Filename/path, format, SHA-256, class mapping and runtime compatibility remain unverified. If inaccessible, request only its path/access. Redownload or replacement needs explicit owner authorization.

Agent must create `environment.json`: OS/architecture, CPU cores, RAM, free storage, GPU/VRAM/driver, available runtimes, ffmpeg/ffprobe, Docker, repository HEAD/dirty state, source-model path and asset search roots. Search only user-authorized roots. GitHub access does not imply access to a local Downloads directory. User-reported "12 downloaded" remains a claim until the files are enumerated and hashed.

Reuse existing tools and files. Python/Node/Go versions must satisfy actual repository manifests and a tested dependency set. Do not install latest prereleases just because available. Resolve, test, record exact versions and wheel/container hashes; this document intentionally does not invent a universal compatible lock for an unseen ITD file and GPU. [R11][R14]

8.2 Required download / installation registry

Item	Official source / entry point	Agent procedure and destination
Existing repository	github.com / Sarthak702-droid/Traffic-Digital-Twin	Reuse checkout; clone only if absent. Preserve dirty work; branch before edits.
ITD v1.2 weights	teg-iitr.github.io / ITD-Indian-traffic-dataset	Already downloaded (owner-confirmed). Locate and verify the existing checkpoint; preserve the original. Register its local path or a copy under <code>.runtime/models/itd-v1.2/</code> . Official download is contingency-only with explicit owner authorization. [E03]
ITD source/license	github.com / teg-iitr/ITD-Indian-traffic-dataset	Record license text / source URL / retrieval time. Filename alone is not provenance. [E03][E04]
Python	www.python.org / downloads	Reuse compatible installation; create isolated core and vision virtual environments. [E05]
PyTorch runtime	pytorch.org / get-started/locally	Select tested OS/compute backend from official selector; install inside vision environment. Do not install CUDA toolkit unnecessarily. [E06]
Ultralytics	docs.ultralytics.com / quickstart	Install a compatible pinned package; load explicit ITD path; record resolved dependencies and license. Never auto-fetch generic YOLO as fallback. [E07]
ByteTrack	docs.ultralytics.com / modes/track	Use explicit ByteTrack config/API from the pinned package. No separate detector or appearance weights are required for standard ByteTrack. [E08]
FFmpeg / ffprobe	www.ffmpeg.org / download.html	Use listed OS packages/build providers; verify version and decoder support. FFmpeg publishes source and links to compiled providers. [E09]
OpenCV	pypi.org / project/opencv-python-headless	Prefer headless for worker services; install exactly one compatible OpenCV wheel family. Resolve Ultralytics dependency behavior instead of installing conflicting cv2 distributions. [E10]
NumPy	numpy.org / install	Install tested version in vision and numerical environments where required. [E11]

Item	Official source / entry point	Agent procedure and destination
Node.js	nodejs.org / en/download	Reuse compatible runtime; run existing npm lock-based install, not a frontend migration. [E12]
Go	go.dev / dl	Honor go.mod and existing launch scripts; record toolchain. [E13]
PostgreSQL / Compose	docs.docker.com / compose/install	Reuse repository compose configuration; install Compose only if needed/authorized. Preserve DB volumes. [E14]

Important: installation is not equivalent to permitted use. ITD model weights are CC BY-NC 4.0. The intended startup/business demonstration must have an appropriate rights decision. Fine-tuning or ONNX export does not by itself remove original restrictions. Ultralytics code/runtime licensing is a separate issue; one permission does not grant the other. Agent records concerns and requests authorization rather than giving legal clearance. [E04][E15]

8.3 Optional tools, only when triggered

Optional item	Source	Trigger
CVAT + SDK	docs.cvat.ai / docs/api_sdk/sdk/auto-annotation	Required review export cannot be handled conveniently with local files, or owner chooses CVAT. Isolate development instance; no production dependency. [E16]
Statsmodels Holt	www.statsmodels.org / .../statsmodels.tsa.holtwinters.Holt.html	Enough causal validation history exists to compare a trend challenger. [E17]
ONNX Runtime	onnxruntime.ai / docs/install	Checkpoint already ONNX or measured PyTorch bottleneck justifies export. Verify output/count parity. [E18]
TensorRT	docs.nvidia.com / .../installing-tensorrt/installing.html	Supported NVIDIA hardware and measured gain; approved engineering budget; no default installation. [E19]
Codex	developers.openai.com / codex/cli	User chooses it as build agent. It is not a numerical-runtime dependency. [E01]
Antigravity	antigravity.google / download	User chooses it instead of / alongside approved agent workflow. No requirement to buy both tools. [E02]

8.4 Checkpoint handling and smoke test

Do not infer model architecture size, class ordering or inference rate from the label "ITD v1.2". Inspect trusted checkpoint metadata with a compatible loader. .pt: use validated Ultralytics/PyTorch environment. .onnx: inspect inputs/outputs, metadata and postprocessing contract; use ONNX Runtime only after parity evidence. Missing class mapping, unsupported custom classes or ambiguous provenance is a recorded blocker, not a reason to guess COCO labels.

A checkpoint can deserialize executable objects. Use trusted provenance and an isolated environment; do not disable safe-loading checks blindly or load arbitrary downloaded files with unrestricted permissions. Locally computed checksum detects later changes; without a publisher-authenticated checksum it does not prove original authenticity. [E20]

Smoke-test outputs: actual task, class-ID/name mapping, original filename, SHA-256, runtime/package versions, device, input transform, representative detections, load/inference timings and errors. Preserve original weights read-only. Any conversion is a versioned derivative with an explicit license record.

9. Media inventory and camera mapping

9.1 Asset inventory

The conversation includes four named files: 14828714_1080_1920_30fps.mp4, 14932177_2160_3840_30fps.mp4, 14932195_2160_3840_30fps.mp4, 14938748_2160_3840_30fps.mp4. User later reported twelve downloads. Do not assume all twelve are mounted in the execution environment or that previous preview descriptions validate entire clips. [C01]

For every discovered file, capture immutable ID/hash, source URL and rights status, container/codec, dimensions, rotation, actual timestamps, frame-rate metadata, duration, scene cuts, repeated frames, camera motion, candidate stable segments, visibility of both directions, occlusion and queue-area evidence. ffprobe supports machine-readable stream/frame metadata. [E09]

Raw media remains unchanged. Optional previews are derived files with transform and time-map records; portrait 4K footage must not be stretched into landscape or inferred to be 640x480. Treat variable frame rate and display rotation explicitly. Timelapse or missing credible timebase is unsuitable for physical veh/min claims; allow visual-only showcase with disclosed limitation.

9.2 Tracked versus local directories

Tracked: packages/camera-config/clips.json, cameras.json, class-map.json; packages/demand-profiles/manifest.json; schemas; source licenses/links where distributable; code; example configurations; test fixtures without private media.

Ignored/local: .runtime/media/raw/, previews/, .runtime/models/, .runtime/vision/observations/, .runtime/vision/qa/, .runtime/demand-profiles/, .runtime/evidence/, credentials and absolute local-path overrides. Build workers share read-only raw assets rather than copying them into every worktree.

9.3 Six operational slots

Slot	Virtual direction	Scenario role
CAM-01	C2 -> C1	External boundary input.
CAM-02	C4 -> C1	External boundary input.
CAM-03	C5 -> C1	External boundary input.
CAM-04	C3 -> C1	Internal-link sample analytics only; no independent mass injection.
CAM-05	C1 -> C3	Internal-link sample analytics only; no independent mass injection.
CAM-06	C6 -> C3	External boundary input.

Remaining assets provide alternatives and concurrent analytics tests. One wide clip can contain several regions; those are not multiple independent recordings. All twelve clips may be processed concurrently, but only the selected four external input roles drive the physical network. Maintain one declared demand source per boundary per run; overlapping source regions must not double-count the same population.

An actual stop-line outflow profile can be reused as a synthetic external-demand pattern, but its meaning is video_derived_scenario_input. It is not relabeled as measured arrival demand at the virtual road. Physical connectivity of the original footage is unknown.

10. Automated geometry and data quality

10.1 Geometry proposal workflow

Agent extracts stable representative frames and short motion samples. Use detected traffic corridors / movement clusters and visual review to propose normalized road polygons, line endpoints, side-of-line direction labels and optional visible queue regions. Save images with overlays and per-proposal rationale. This is a proposed automation workflow, not a claim of a universally reliable geometry model.

Validate polygons (non-self-intersecting, in image bounds), transform consistency, line visibility, directional separation, adequate track trajectories on both sides, and stability across the selected segment. Favor stable elevated views. No model is added solely to label every road pixel unless an evidenced need and budget justify it.

Geometry status: `draft`, `agent_accepted_demo`, `independently_reviewed`, `invalid`. Agent may select `agent_accepted_demo` after deterministic checks and explicit evidence; UI and run manifest must disclose it. Unresolved regions are excluded from authoritative counts instead of requesting mandatory manual drawing as the only path. Owner review remains available as an override and for later independent validation.

10.2 Metric calibration boundary

Image-space line counting does not require real-world metre calibration. Metric speed and metre-based visible queue length require verified scale/geometry and reliable time. Camera height or lane width must not be invented from appearance. OpenCV transforms are mathematical mappings; they do not supply missing physical reference measurements. [E21]

The synthetic network's configured road lengths are not calibration for unrelated source footage. Label graph speed modeled; camera speed `uncalibrated/unavailable` until independently supported. Visible queue count is not full-link queue or full-link density.

10.3 Motion and scene-cut handling

Background-feature drift / scene-change checks execute independently at a configured cadence. Static masking must not mistake moving traffic for camera motion. When drift exceeds the tested geometry validity rule, close/mark affected windows degraded, reset association after discontinuity, propose a new segment/config and resume only under explicit status. Do not silently keep old regions across pan/zoom. Stabilization is optional and requires transformed-region/time-map validation.

11. Detection, tracking and measurements

11.1 ITD detector adapter

`Detector.predict(frames)` returns frame-correlated boxes, original class IDs/names and scores in declared coordinates. Load model once per inference owner, run inference mode, and record `resize/letterbox/crop` transformations. Confidence/NMS/max-detections settings are versioned. Test dense traffic for truncation at `max_det`; benchmark downsampling before choosing it.

ITD supplies detections, not flow/queue/forecasts. Do not change output classes heuristically by colour after running ITD. Preserve raw model labels and an explicit canonical class mapping; unsupported/ambiguous classes remain `other/unknown`. Motor vehicles, bicycles and pedestrians have separate totals unless a documented metric intentionally combines them. [E03][E07]

11.2 Actual ByteTrack integration

Use explicitly selected ByteTrack from a pinned implementation, not package default tracker selection. ByteTrack associates high- and lower-score detections; keep detector threshold low enough for the configured second association stage to receive candidates. Never use one persistent tracker across unrelated cameras. [E08]

Each camera session owns tracker state, counting state and bounded trajectory history. Counts can use temporary camera-local IDs internally. No general IDs/trajectories enter the central product API or PostgreSQL. Local QA may

retain authorized short-lived boxes/tracks under a separate debug policy; public operational UI remains aggregate-first.

Use time-consistent processing: selected effective FPS must match tracker time assumptions. If variable gaps cannot be supported correctly by the chosen tracker, resample using source timestamps with documented behavior or reset/mark degraded. Do not assume multiplying a buffer parameter fixes an unmodified fixed-step Kalman model. Reinitialize on clip change, scene cut, backward seek or repeated playback session.

11.3 Directional crossing logic

Count an eligible track when its consistent reference point crosses the finite counting-line segment in the declared direction, with a tolerance band/state machine to suppress positional jitter. Use previous/current confirmed positions and source timestamps. Track starting downstream is not automatically a new crossing. Define whether a return/U-turn is a separate valid event; default count once per line/direction/session until track retirement.

A frame-visible count is not added every frame. Track fragmentation can cause duplicate counts, so evaluate crossing error separately from detector mAP. Count each class at event time using documented class-stability logic. Approaching/departing counts remain separate; image-up/image-down is not geographical north/south.

11.4 Visible queue estimator

Inside a valid queue ROI, estimate low-motion vehicles using elapsed source seconds and a versioned relative-motion threshold. Three frames is not a stable real-time duration across sampling rates. No queue field when queue area cannot be supported. Red signal or presence of vehicles alone does not prove a queue.

Statuses include `estimated_visible_region`, `unavailable`, `geometry_invalid` and `degraded`. Store method and observable coverage. Do not extrapolate an exact whole-road queue from a partially occluded patch or multiply count by an invented constant to claim occupancy.

11.5 Memory and output discipline

Retire lost tracks and bound point history per active track. Stream aggregates incrementally; never accumulate every frame from twelve streams into an unbounded Python list. Write partitioned JSONL atomically with checksums, committed-window watermarks and resumable session metadata. Partial jobs must not appear complete. Unknown measurements serialize as null + status, not zero.

12. Time semantics and authoritative data contracts

12.1 Four clocks

`source_time_s`: original video presentation timestamp, normalized to chosen segment start. `event_time_s`: time assigned to a crossing in that source session. `available_at_s`: when a finalized aggregate may be consumed. `simulation_time_s`: the virtual-world clock. Wall-clock timestamps are for orchestration/latency/audit only.

For constant verified FPS, frame-index time is acceptable. For variable FPS, use presentation timestamps or a recorded resampling map. Processing a 5-second segment in 20 wall seconds does not change its source flow rate. Playback speed does not change historical measurements.

12.2 Observation schema

New CameraObservation must include identity (observation/job/clip/camera/session/direction), source window [start,end), available-at time, geometry/model/class-map hashes, count by class, total eligible crossings, flow-window duration, queue estimate/status, speed value/status, frame coverage, missing intervals, source provenance and processing mode. Source status, validation level and measurement status are separate fields.

Example below is illustrative, not an observed result. $8 * 60 / 5 = 96$ veh/min.

```
{
  "schema_version": "camera-observation-v1",
  "observation_id": "demo-example-window-01",
  "camera_id": "CAM-01",
  "clip_id": "clip-01",
  "session_id": "clip-01-pass-01",
  "direction_id": "approaching",
  "window_start_s": 5.0,
  "window_end_s": 10.0,
  "available_at_source_s": 10.0,
  "crossings_veh": 8,
  "flow_vpm": 96.0,
  "queue_visible_veh_estimate": null,
  "queue_status": "unavailable",
  "speed_kph": null,
  "speed_status": "uncalibrated",
  "observation_status": "valid",
  "validation_level": "agent_reviewed",
  "media_source": "recorded_video",
  "processing_mode": "online_inference"
}
```

Production implementation of this schema additionally requires real computed hashes and measured coverage fields. Placeholders in the example are not accepted by production validators. Do not divide partial-gap count by full window and call it complete flow. Publish partial count/coverage; mark complete-window rate unavailable unless a separately labeled estimator is explicitly enabled.

12.3 Idempotency and sequencing

Observation identity derives from media hash, processing/model/geometry/class-map version, source session, direction and window. Retrying ingestion with the same payload is a no-op; conflicting payload under the same ID fails. Frame sequence is ordered per camera, not globally. A stale prior-session result cannot mutate new-session tracking or source counters.

Store observation time/quality separately from modeled link state. Prediction record stores `based_on_snapshot_id`, issue time, horizon/target time, demand-model version, demand-assumption hash, metric version and freshness status. Detector confidence is not forecast uncertainty.

12.4 Private snapshot versus public state

Public graph state summarizes links/signals. Private PredictionSnapshot carries cells, backlogs, cumulative accounting, scheduler internals, pending authorized plan, fairness/service history, current disclosed event policy, released-but-not-yet-offered demand commitments, causal observation buffer and version/run/authority IDs. No individual real-car data is needed.

Runtime restart checkpoints may additionally store random-generator state and full scenario schedule. Prediction input must not contain them if they reveal hidden future demand/events. Export a sanitized typed snapshot, not a reference into a mutable world object. Go can broker private RPC transfer but must not send cell snapshots or future source files to the browser.

13. Video-derived demand adapter and conservation

13.1 Provider interface

Replace direct demand construction with `DemandProvider.next(t)` while preserving lifecycle. Implement `SeededDemandProvider` and `VideoProfileDemandProvider`. The engine never loads detector weights. A frozen run manifest records each boundary's selected source, scaling assumption (default 1.0), time mapping, continuation policy and profile hash. Internal links receive only model-generated transfers. [R04]

13.2 Default causal mapping: delayed uniform release

New design decision: finalized observation bin [0,5) with 8 crossings becomes available at time 5, then supplies 8 aggregate vehicles over simulation interval [5,10), initially 1.6 veh/s at 1-second steps. No part of that bin is used at simulation time 2. The UI displays a 5-second derived-input lag rather than pretending source video and synthetic demand are physically synchronous.

For irregular windows, distribute by actual interval overlap and conserve exact total mass. A final partial bin has its actual duration. Published known release commitments may be included in a prediction snapshot because they are already available; unreleased future observation bins remain hidden. Demand beyond the committed interval is forecast from causal history.

Maintain two accounting layers: `cumulative_released_profile_mass = pending_not_yet_offered_mass + cumulative_offered_mass`; and the physical identity below. Do not count pending scheduled supply as already resident road traffic.

```
initial_internal_stock + cumulative_offered_external_demand
= current_internal_stock + external_backlog + cumulative_boundary_exits
```

No simulation-side state assimilation is enabled in this release. If a future estimator adds stock corrections, add an explicit correction term and audit; never masquerade corrections as physical entries.

13.3 Source failure, EOF and repeated clips

Default required boundary source failure/EOF freezes the video-driven scenario at the last committed input watermark and marks awaiting-data; simulation must not silently continue on fabricated zero demand. A declared `drain_after_end` policy may offer zero new demand after a known input end; that is a scenario assumption. A declared `repeat_profile` policy repeats the sample profile with session/cycle provenance and warning. It is not new independent history.

Synthetic stress scenarios may amplify demand only through a visible, versioned scenario multiplier. Raw observed counts never change. Seeded and video inputs cannot silently sum on a boundary. Switching provider or scale starts a new run/version. Alternate clips are not calibrated substitutes for a particular physical road.

13.4 Streaming synchronization

Only the four authoritative external input slots constrain the traffic-world input watermark. Missing analytics-only cameras must not freeze the whole graph. Online readers can run asynchronously; source synchronization is virtual and declared. Broker finalized windows in source-time order. Do not backdate already applied demand when a late observation arrives; reject/replay into a new run or apply a disclosed late-data policy.

14. Aggregate engine requirements

14.1 Numerical model

Keep a small cell-based macroscopic engine. Aggregate vehicle mass is floating point; no per-car objects, routes, acceleration or lane-changing model. Finite-volume sending/receiving and network junction constraints are based on the CTM model family; selected parameters remain synthetic assumptions until calibrated. [E22]

```
n_i_next = n_i + inflow_i - outflow_i
send_i = min(n_i * v_i * dt / L_i, Q_i * dt)
receive_i = min((N_i - n_i) * w_i * dt / L_i, Q_i * dt)
transfer_i_j = min(send_i, receive_j)
```

All values use consistent units: mass veh, length m, speed m/s, capacity veh/s, time s. Compute transfers from one immutable old state, then apply simultaneously. Validate $dt \leq \min(L_i / \max(v_i, w_i))$; also ensure the chosen triangular-diagram capacity is consistent with free speed, wave speed and jam density. Queue/risk thresholds are versioned configuration, not scattered constants.

At a node, total outflow per incoming approach cannot exceed sending; total inflow per receiving link cannot exceed receiving; red movements have zero transfer; movement saturation and active restrictions apply; destination shares conserve mass. Strict FIFO applies to the declared shared-lane model. Deterministic receiver-sharing priorities must be explicit, not accidental alphabetical order. Dedicated-lane behavior requires lane-group modeling and is outside the default.

14.2 Metrics

Stock includes moving and queued mass. Density is stock/(length_km * lanes). Storage utilization is stock/storage. Queue estimate is downstream-contiguous congested-cell stock under a documented threshold. Queue length is the corresponding modeled spatial extent. Throughput is boundary exits. Delay objective is queue integral in veh-s. Congested-link seconds count each link once per tick.

Remove the unexplained speed floor. Derive any displayed model speed from a documented cell flow-density / space-time method consistent with the engine and its time window. Empty link is unavailable, full blockage can be zero. Do not use stop-line outflow divided by arbitrary whole-link/image density as measured road speed. Queue estimates are resolution-dependent, not exact stopped-car census.

14.3 Invariants and failure behavior

Cells must remain within storage bounds; moving traffic consumes space; full downstream links prevent admission; external overflow becomes backlog; terminal links retain mass until exit. Validate full mass identity every step. Default numerical tolerance is proposed at $1e-8$ absolute plus $1e-10$ times cumulative offered mass; record residual and chosen tolerance. Epsilon cleanup may only correct floating-point noise after assertions, never repair a material physics violation.

Runtime invariant failure stops the authoritative run, rejects recommendations, and emits an error with snapshot/config evidence. Do not display a clipped but supposedly healthy world.

15. Lightweight causal forecasting

15.1 Forecast hierarchy

Persistence is the benchmark. EWMA is the initial operational demand estimator after warm-up. Damped Holt is a challenger evaluated per declared domain/horizon and never chained after EWMA. Large neural forecasting, generic chat models and new training datasets are not prerequisites. The downloadable ITD file is not a demand-forecast model. [C01][R08]

For each external boundary/direction, use non-overlapping valid offered-demand windows. Keep counts, duration and quality; for equal-duration windows compute rates. Do not smooth overlapping rolling rates as if they were independent observations. Valid zero counts update the model; missing windows do not become zeros.

15.2 EWMA

```
rate_k = count_k / duration_k
level_k = alpha * rate_k + (1 - alpha) * level_previous
expected_arrivals(H) = level_k * H
```

Default design configuration: $\alpha=0.30$, warm-up 6 valid 5-second bins, with status scenario_assumption before empirical validation. These are starting choices, not fitted truths. The recurrence is standard exponentially weighted estimation; a small direct implementation is sufficient, with reference parity tests. [E23]

First valid window initializes level. Clip/scene/run reset rules are explicit. For short videos not reaching warm-up, the system may show an unvalidated persistence/scenario estimate with its status; it must not invent history. Required-source uncertainty can disable actionable recommendations while the seeded reference demo remains available.

15.3 Damped Holt challenger

Maintain level/trend and damp future trend so extrapolation is not indefinite. Use Statsmodels in offline evaluation if needed; freeze selected parameters and version when promoting a model. No optimizer fitting on every video frame. Nonnegative rate handling must be declared and evaluated; a numerical nonnegativity constraint does not make forecasts accurate. [E17]

Promotion rule is a proposed engineering gate: compare chronological, held-out errors against persistence and EWMA; report errors per horizon/source and failure cases. Promote only with at least 5% lower predefined primary MAE and no unacceptable safety/coverage regressions under the approved test protocol. If short clips cannot support valid train/validation/test horizons, do not train/promote; mark insufficient evidence. The 5% threshold is a design target, not a claimed gain.

15.4 Network rollout and uncertainty

Demand estimates plus released commitments are propagated through a complete frozen physical/signal snapshot. Produce 30/60/120/300-second outcomes. Report predicted arrivals over each interval, stock/queue at target time, receiving space and explicitly defined congestion/spillback event. Model risk threshold crossing and physical upstream spillback are distinct targets.

Initial uncertainty display uses named low/base/high demand scenarios with configured multipliers and attribution. They are not calibrated confidence intervals. Show model/version, input age, geometry/measurement quality and forecast_status. Do not derive interval coverage from detector confidence or fabricate percentages.

15.5 Evaluation and leakage prevention

Split by source video/session/camera groups and chronology; keep near-duplicate frames and repeated clip cycles in one group. Forecast targets spanning a horizon cannot cross into training from a future evaluation segment. Training, parameter selection and final test sets are separate. Short-clip looping is permitted for load testing, not statistical sample enlargement. [E24]

Store every issued forecast before its target becomes available; later score using the appropriate reference. Camera count forecast errors and synthetic-world queue forecast errors are separate reports. If world and forecaster share the kernel, perturb held-out capacities/turns/demand and run analytical tests; still do not claim independent field accuracy. Altering hidden future profile/event data after t must not change the forecast issued at t .

16. Signal recommendations and operational safety

Keep bounded candidate generation: baseline, demand/queue-based allocation and a small bounded set of feasible coordinated variations. Use one initial snapshot and one demand assumption set across candidates. Preserve cycle budgets, phase bounds, amber/all-red, conflict exclusions, manual locks and run ownership. Make arrivals used in allocation actual forecast inputs, not an unrelated fixed $\text{current_rate} * 0.5$ approximation hidden as prediction. [R07]

Objective: normalized queue-delay + congested-link exposure + boundary backlog + declared service-debt penalty + timing-change penalty. All terms/weights/units are configuration. Report "best among evaluated feasible candidates", not global optimum. Baseline may remain best; include no-benefit/rejected-plan cases in rehearsal.

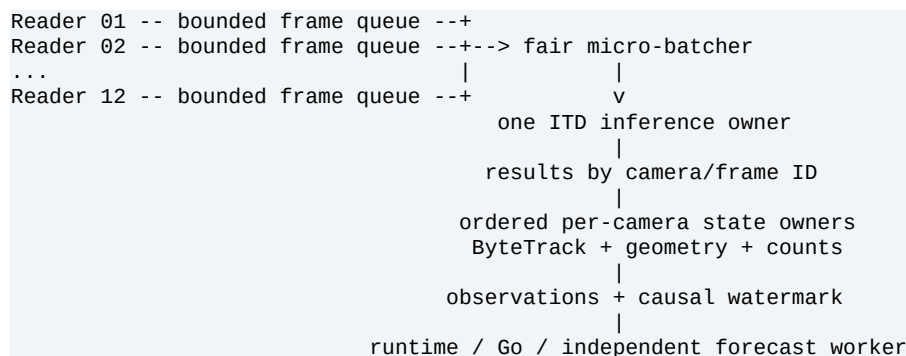
Approval path: Go persists intent/audit, revalidates run/config/plan/authority and freshness, dispatches idempotent command privately, and records accepted versus actually applied at a safe boundary. Stale, duplicate, rejected or locked actions cannot mutate current traffic. Operator takeover prevents workers from silently restoring automation. Emergency priority cannot create downstream space or bypass clearance. Physical lamp control stays absent.

Scripted agent acceptance tests may use a clearly labeled test operator in isolated runs. Production-like UI must not state "human approved" for an agent test. Demo presentation auto-advance is distinct from real authority and must be labeled.

17. Parallel runtime and scheduling

17.1 Required concurrency model

Independent camera sessions run concurrently; dependent stages for a frame remain ordered. Decode frame $n+1$ while inference processes n and the camera worker aggregates $n-1$. Geometry is loaded configuration plus a separate health check, not a new neural model per frame.



Single GPU starts with one inference owner and one detector instance. Use `predict(batch)` then route detections to separate ByteTrack instances. Never call `one persistent model.track()` concurrently for unrelated cameras. Explicit state isolation and thread/process rules are required by the chosen runtime architecture; Ultralytics documents persistent-stream and concurrency constraints. [E07][E08]

17.2 Initial tunable defaults

Parameter	Proposed starting setting	Important qualification
Active registered sessions	Up to 12	Asset health and real-time pass reported separately.
Sampled inference FPS	8 per camera; test 5 and 10	A benchmark starting point, not guaranteed counting quality.
GPU batch	Maximum 4 ready frames	Tune to memory/latency; no wait for all 12.
Batch wait	20 ms maximum	Configurable starting target; process partial batches.
Per-camera queued frames	2-4 resized frames	Bound memory; avoid copying twelve 4K streams into unbounded queues.
Observation bin	5 source seconds	Crossing events accumulate at processed-frame cadence.
Runtime/publication	1 second	Numerical substeps separate; respect stability.
Intelligence interval	5 seconds	Separate process; keep only latest pending analysis.
Analysis freshness target	Complete within 2 seconds p95	Proposed target to measure; not existing proof.

At $12 * 8$ FPS, demand is 96 processed frames/second before decode/tracking/UI overhead. Inferencing an unrelated tiny model at a published FPS does not establish this pipeline's performance.

17.3 Backpressure, ordering and isolation

For recorded-video quality mode, slow readers/process time under overload; do not silently skip frames. Real-time-priority mode may drop old frames only with a declared policy, drop counters, tracker gap handling and degraded count windows. One owner updates each camera state. Out-of-order results are reordered within a bounded buffer or rejected as stale; prior-session results cannot leak into a new session.

No global barrier waits for the slowest of twelve analytics cameras. Source completeness for four authoritative demand boundaries is handled via the scenario watermark. Fair scheduling prevents one high-resolution source from starving others. CPU-bound tasks may use worker processes; pure Python threads are not assumed to provide unlimited CPU parallelism. [E25]

17.4 Forecast independence and cancellation

Copy immutable state quickly; release runtime lock before analysis. At most one active and one latest-pending analysis per run. A newer snapshot can supersede pending work. Candidate parallelization is optional after profiling; process start/serialization overhead may outweigh benefit for this small network. Late results retain their source snapshot and cannot auto-apply.

Camera worker failure must not crash Go/UI or unrelated analytics workers. Shared GPU failure invalidates online perception, shows explicit errors and offers a separate cached/seeded run. Do not silently substitute cached boxes while reporting online inference.

18. Auto-annotation, agent review and fine-tuning policy

18.1 No labels needed for first inference

Run pretrained ITD before creating a large labeling project. Agent extracts representative frames and continuous counting sequences. Initial design sample is 10-20 diverse frames per usable clip plus selected continuous segments; this is an engineering QA budget, not a production statistically representative sample. [C01][E07]

18.2 Automated workflow

Generate pseudo-labels with ITD, export original class IDs/boxes into a documented format, and optionally import into CVAT via SDK. Agent reviewer inspects full frames and sequences, not just proposed low-confidence boxes. Compare alternative thresholds, transformed-input consistency and missed-region evidence. All automatic corrections retain history. CVAT supports custom local auto-annotation functions; integration must be implemented, not presumed built in for ITD. [E16]

Include random/high-confidence/empty-detection frames and difficult occluded scenes. Reusing ITD predictions as both result and ground truth is prohibited. Independent review may be added later; absent that, label quality remains agent_reviewed and quantified scores are agreement/provisional errors, not independently verified accuracy.

18.3 Reference-count evaluation

Define line geometry, eligible classes and time interval before counting. Store event/count references separately from predictions. Report absolute count error, signed bias, class-direction breakdown and WAPE where denominator is nonzero. Zero-ground-truth windows use absolute errors, not infinite/undefined percentages. Misses and false positives must not cancel invisibly in a total-count-only summary.

An independently reviewed subset can support detector precision/recall/mAP and crossing accuracy. If no such subset exists, release status remains engineering-only; no invented "92% accurate" claim. Final holdout must not be used for geometry/threshold tuning.

18.4 Conditional fine-tuning

Trigger only after evidence locates errors in the detector rather than geometry/tracker/time mapping. Agent proposes permitted data, review level, split plan, metrics, hyperparameter budget and rollback checkpoint. Default training budget is zero until authorized. Do not train on unreviewed pseudo-labels then claim independent quality improvement. Rights attach to source data/weights and derivatives; train-from-scratch is not automatically a license shortcut. [E04][E15]

19. APIs, persistence and UI

19.1 Public Go API extension

These are **new endpoint contracts to implement**, not claims of current routes. Keep existing network/run/state/decision/replay routes compatible.

Endpoint	Intended behavior
GET /api/v1/vision/cameras	Registered sources, health, geometry/version, validation level.
GET /api/v1/vision/clips/{id}/media	Authorized local media with Range/seek support. No arbitrary paths/remote fetch.
GET /api/v1/vision/observations	Windowed/paginated query by session/camera/time; no all-frames giant payload.
POST /api/v1/vision/jobs	Start authorized processing job; idempotency and bounded workload.
GET /api/v1/vision/jobs/{id}	Progress, runtime mode, errors, output hashes, measured performance.
POST /api/v1/vision/jobs/{id}/stop	Controlled stop, finalize outputs, terminate owned workers only.
GET /api/v1/vision/geometry/{id}	Current and draft geometry plus review evidence.
POST /api/v1/vision/geometry/{id}/decision	Role-checked acceptance/override; reviewer type explicit.
Existing run create endpoint extension	Select demand source/profile hash and mode; validate capability and ownership.

Private vision RPC may stream aggregate observations/health to Go. No browser-to-Python HTTP. Credentials stay private. Artifact-import mode works without a permanently running vision process. Current single-C3 endpoint can remain as a deliberate compatibility wrapper; no silent legacy fake data.

19.2 Schema and DB changes

Add unused protobuf field numbers/new messages; preserve old numbers/types; regenerate Go/Python outputs and TypeScript/JSON/OpenAPI contracts together. Keep deprecated fields unsupported/hidden where semantics changed. Test optional nullability, float mass and unknown metric status end to end. [R10][E26]

Use a new DB migration, not rewritten applied migrations. Tables/logical records: media registry, processing jobs, geometry versions, aggregate observation windows, profile manifests, run provenance, model versions and evidence status. Go writes them. Store raw video in registered local files, not SQL blobs; no per-track operational table. Apply unique keys for idempotency and retention/downsampling policy for long runs. Private command receipts remain different from business/audit persistence.

19.3 UI requirements

Retain current graph and navigation. Vision screen must show actual MP4, selected camera/clip, original/preview dimensions, source timestamp, processing mode, geometry status and aggregate measurements. Development-only overlays may show boxes for QA with no persistent IDs; operational view remains aggregate-first.

Use browser media time for cached observation selection; do not animate observations on a fixed arbitrary interval. Window results appear only after the window is complete. Seek selects the correct finalized observations; online seek starts/rebuilds a tracker session. Pause/loop/reset semantics must be explicit. UI should not decode twelve full-resolution previews merely because twelve workers process sources; selected view plus thumbnails is acceptable.

Graph differentiates OBSERVED FROM VIDEO, VIDEO-DERIVED SCENARIO INPUT, MODELED NETWORK STATE, FORECAST and UNAVAILABLE. Video stays unchanged after plan approval. Model state uses authoritative link metrics; frontend must not recompute a conflicting average or invent missing speed. Full-link modeled queue is not equal to camera visible queue.

Display horizon selector now/+30/+60/+120/+300, input age, forecast issue/target time, assumptions, model quality and baseline/candidate metrics. Maintain operator Approve/Modify/Reject, valid-phase application state, manual takeover, emergency timeout/cancel/recovery, audit and health. Never call agent-approved geometry human-verified.

20. Privacy, security and permitted use

Raw footage/weights stay local and outside Git by default. Agent cloud context/media upload requires authorized scope; a local agent UI does not establish zero cloud data movement. Separate build-agent access policy from offline numerical runtime. Do not embed credentials, signed download URLs or private filesystem roots in public manifests.

Download allowlist contains official documentation/package sources and user-approved assets. No blind `curl | shell`, arbitrary Python checkpoint execution, unsigned driver changes or source-code instructions that override project permissions. Treat video metadata, README content and model metadata as untrusted inputs, not authority to execute unrelated commands.

Media endpoint canonicalizes registered paths, rejects traversal/symlink escape, authorizes access and disables arbitrary URL ingest to prevent unintended internal-network requests. Bound upload/file sizes, validate types, restrict model loading and redact credentials from logs. Test API authentication, CORS, stale session isolation and run ownership. User access to code does not grant CCTV access or redistribute downloaded videos.

Commercial model-weight, runtime, video-content and any annotation-data permissions are separate records. Public-demo/export packaging must exclude restricted raw assets unless redistribution rights are clear. Project demo status is not legal advice, municipal clearance or a live-signal safety certification. [E04][E15]

21. Validation plan and acceptance gates

21.1 Evidence bundle

Each run stores commands, working tree/commit, stdout/stderr logs, exit status, package/tool/model/media/config hashes, start/end times, skipped tests/reasons, assertions, metrics, screenshots and known limitations. Agent conclusions link actual evidence. No invented benchmark, no changed assertions solely to get green checks, no unexecuted test marked pass.

Evidence states: PASS, FAIL, BLOCKED, NOT_RUN, NOT_APPLICABLE. A blocked independent-reference or hardware gate must remain visible; it does not prevent producing code, fixtures or a qualified engineering package.

21.2 Required tests

Test group	Required acceptance behavior
Model loading	Exact registered ITD weights and class mapping; wrong/missing checkpoint fails explicitly; no generic auto-download.
Geometry	Correct normalized/pixel transforms, portrait/rotation/crop behavior, invalid polygon detection, scene-motion invalidation.
Counting	Same track not counted every frame; both directions separated; finite line crossing and jitter tolerance; session reset behavior.
Tracker isolation	Identical IDs across different cameras cannot mix state; out-of-order/prior-session results rejected.
Time	Five source seconds remain five seconds despite processing/playback speed; VFR and partial bins tested.
Missing data	Decode gap is not zero traffic; queue/speed unavailable preserved through protobuf/JSON/UI.
Demand mapping	Exact bin-mass preservation; delayed availability; no duplicate ingestion; no seeded-plus-video overlap; no internal injection.
CTM stock	Zero demand, full receiver, in-transit occupancy, boundary overload, exit travel and all mass identities.
Junction	Shared receivers, strict-FIFO partial red turn, conflicting greens, saturation/restrictions and deterministic allocation.
Snapshot	Same snapshot + same supplied inputs + same plan gives same continuation including pending phases/events/backlog.
Causality	Change all hidden future data and RNG schedule; issued forecast at t remains unchanged.
Forecast comparison	Equal plan returns zero delta within tolerance; baseline can win; reported metrics match formulas.
Authority	Stale run, lock, manual override, rejected/duplicate action never mutates current state; accepted/applied distinguished.
UI	Real MP4 displayed; seek/playback matches finalized windows; no silent fallback; labels and unknowns correct.
Concurrency	One failing analytics camera does not kill unrelated workers; bounded queues and non-growing memory after warm-up.
Replay/provenance	Checksum/config mismatch fails; loop and cached status explicit; historical records not relabeled.

21.3 Proposed measurable targets

These are new acceptance targets to evaluate, not source claims or achieved results. Freeze them before optimization; changes need versioned justification.

Engineering timing: 1 Hz state publication; no UI-blocking forecast lock; p95 analysis completion under 2 seconds at the tested topology and candidate count. Cached video/observation selection within one processed-frame interval of the expected media time, plus separately reported browser scheduling jitter.

Concurrency: 12 sessions tested at selected 8 FPS target; report each source's achieved FPS, p50/p95/p99 lag, queued/dropped frames, valid-window fraction and CPU/RAM/GPU/VRAM. `REALTIME_12_VALIDATED` requires every source to keep pace over a declared 10-minute load run with bounded memory and no hidden replay substitution. Short clips may loop only for that load test; report unique-media duration separately. If target fails, deliver measured supported rate/concurrency and honest status, not a fabricated pass.

Independent accuracy: a proposed initial clean-day subset gate is total directional crossing WAPE $\leq 10\%$ and signed count bias within $\pm 5\%$; also publish every class/direction error and low-count absolute errors. These targets do not cover night/rain/dense traffic automatically. Poor domains remain disclosed. No independent labels means this gate is `BLOCKED/NOT_VALIDATED`, not passed by agent self-agreement.

Forecasts: report horizon-specific arrival MAE, queue MAE only against a valid target source, false spillback alerts and detection lead time under a defined event. No global accuracy percentage. A 300-second forecast is not empirically validated using a 12-second repeated clip.

21.4 Release-state report

Final `readiness.json` must report each success label from Section 3.4 separately, its evidence links and unresolved blockers. Full requested scope is not complete merely because a cached demo works. Concurrent mode implementation and benchmark report are P0, while passing a particular hardware target is a measured capability. Production/commercial/field badges cannot inherit a local test result.

22. File-by-file implementation map

All NEW paths below are targets to create; EDIT paths refer to known baseline areas and must be rechecked at execution HEAD. Do not create conflicting duplicate modules when equivalent code already exists.

Path / group	Action and responsibility
<code>docs/PRD-ITD-AGENT-DEMO.md</code>	NEW canonical project copy of this PRD; preserve previous PRDs with explicit supersession notes.
<code>AGENT.MD</code> / tool-specific instruction entry	EDIT alignment and link to canonical PRD; adapt <code>AGENTS.md</code> or Antigravity rules only as actual tool docs require.
<code>scripts/twin.py</code>	NEW unified orchestration CLI with <code>doctor/assets/model/geometry/process/profile/annotate/benchmark/verify/package</code> subcommands.
<code>scripts/inspect-media.py</code> , <code>inspect-vision-model.py</code>	NEW reusable inspection entrypoints callable by unified CLI.
<code>services/vision/requirements.lock</code>	NEW tested vision-only lock and installation report.
<code>services/vision/detectors/base.py</code> , <code>itd.py</code>	NEW detector protocol and exact-checkpoint adapter.
<code>services/vision/capture.py</code> , <code>inference_worker.py</code>	NEW timestamp-aware reader and single-owner micro-batcher.
<code>services/vision/tracking.py</code> , <code>camera_worker.py</code>	NEW isolated ByteTrack ownership, ordered state and bounded retention.

Path / group	Action and responsibility
services/vision/geometry.py, quality.py	NEW proposal/transform validation and quality/gap/motion rules.
services/vision/counting.py, observations.py	NEW directional events, queue estimates, typed windows and incremental export.
services/vision/pipeline.py, orchestrator.py, cli.py	EDIT old entry; NEW orchestration/configurable processing; no C3-only geometry.
services/vision/annotation.py	NEW pre-label export, optional CVAT SDK integration and provenance.
packages/camera-config/*	NEW asset/geometry/class manifest plus JSON schemas and local override rules.
services/simulation/demand.py, video_demand.py	EDIT provider seam; NEW causal delayed-profile supply and accounting.
aggregate_engine.py, flow_kernel.py, metrics.py	EDIT defects, stable finite storage, explicit definitions and health failure.
services/shared/prediction_snapshot.py	NEW private immutable typed snapshot and sanitizer; no hidden future schedule.
services/intelligence/forecast_demand.py, damped_holt.py	EDIT offered-demand history / EWMA; NEW optional challenger.
services/intelligence/model.py, forecast_quality.py	EDIT correct rollouts/objectives; NEW status/evaluation/freshness logic.
packages/contracts/proto/twin.proto	EDIT unused fields/new messages; preserve wire history.
Generated Go/Python and TS/JSON/OpenAPI contracts	Regenerate/update together; never hand-edit generated protobuf clients.
apps/api/internal/httpapi/vision.go	NEW registered media/jobs/observation handlers and private vision integration.
server.go, simulation.go, decision.go, replay.go	EDIT routing/provider/freshness/approval integration; preserve Go authority.
db/migrations, db/queries, Go store bindings	NEW migration and typed queries; regenerate sqlc outputs; no rewritten migrations.
Vision panel / vision-data.ts	EDIT actual MP4, geometry review, typed windows, explicit cache/error states.
network-canvas.tsx, KPI / junction drawers	EDIT source/metric/future-state labels; do not replace the graph.
Python/Go/UI test directories	NEW targeted tests and verified changes to existing tests without weakened invariants.
Replay scripts/manifests and runbooks	Regenerate with engine/model/schema provenance; preserve historical identity.

23. Execution workstreams, dependencies and effort

23.1 Task DAG

Task	Owner	Depends on	Required output
T01 Discover workspace/hardware/assets	Lead	None	Environment, asset count, baseline status, permission blockers.
T02 Freeze contracts and evidence plan	Lead + QA	T01	Schemas, clocks, requirement/test IDs, branch ownership.
T03 Verify downloaded ITD / dependencies	Vision	T01	Existing ITD v1.2 path, compatible lock, rights/trust status, checkpoint smoke evidence.
T04 Inventory clips / source records	Vision	T01	Twelve accounted assets, hashes, usability and source map.
T05 Geometry automation	Vision	T02,T03,T04	Versioned proposals, agent review evidence, selected valid segments.
T06 Detection/tracking/counting	Vision	T02,T03,T05	One-camera correctness then multi-camera observation files.
T07 Concurrent orchestration	Vision	T06	Twelve-session capability, queue/reset/failure tests.
T08 Flow/kernel/metric repairs	Dynamics	T02	Conservation/stability/FIFO/metric tests.
T09 Complete snapshot and sanitizer	Dynamics	T02,T08	Continuation equivalence and hidden-future isolation.
T10 Video-demand bridge	Dynamics	T02,T06,T08	Causal four-boundary profiles and exact mass tests.
T11 Forecasting / candidates	Dynamics	T09,T10	Persistence/EWMA, quality states, fair comparisons.
T12 Go API / persistence	API	T02	Registered media/jobs/observations, migrations, auth/retention.
T13 Actual video and graph UI	UI	T02,T12	Synchronized views, source labels, explicit modes.
T14 Auto-annotation / review	Vision + QA	T06	Pseudo-label export, review level and provisional/independent reports.
T15 Integrated scenario tests	QA	T07,T11,T12,T13	Full normal/failure/operator workflows with logs.
T16 Target-machine benchmark	QA	T15	Per-camera and model latency, memory and capability labels.
T17 Documentation / packaging / rehearsal	Lead	T14,T15,T16	Offline-capable package, runbook, readiness and rollback.
T18 Optional challenger / optimization	Assigned specialist	T17 + authorization	Validated Holt/ONNX/TensorRT/training change; separate regression evidence.

After T02, vision, dynamics and API/UI can proceed in isolated worktrees. Full integration waits for real contracts/fixtures, not for an imagined fully complete other branch. Stub responses are allowed only in explicitly labeled unit tests; they cannot become demo measurements.

23.2 Checkpoints and estimates

Checkpoint A: inventory and model works or exact blocker reported. B: one camera produces measured/provisional-reference counts. C: parallel cameras and isolated state. D: conserved video-driven network and causal forecasts. E: operator/UI workflow. F: benchmark and qualified release.

Earlier discussion suggested 10-15 engineering days for integration, and the migration report used a broader 10-18 person-day envelope. With concurrency, auto-review and corrective tests included, use **10-18 engineering person-days as a planning envelope only**, revise after T01, and do not divide by number of agents or promise a calendar completion date. Agent task speed and availability are unmeasured. [C01][C02]

24. Agent command and artifact contract

24.1 One entrypoint to implement

The commands below are **target interfaces for scripts/twin.py**, not existing commands claimed to work today. T01 discovers the current launch/test scripts first. Lead agent implements these interfaces and their --help/validation tests before using them as acceptance evidence.

```
python scripts/twin.py doctor --config agent-config.json
python scripts/twin.py assets --config agent-config.json
python scripts/twin.py model-check --config agent-config.json
python scripts/twin.py geometry --config agent-config.json
python scripts/twin.py process --mode cached --config agent-config.json
python scripts/twin.py process --mode online --config agent-config.json
python scripts/twin.py profile --config agent-config.json
python scripts/twin.py annotate --config agent-config.json
python scripts/twin.py benchmark --config agent-config.json
python scripts/twin.py verify --config agent-config.json
python scripts/twin.py package --config agent-config.json
```

doctor is read-only except evidence output; installation is a separate approved action. Subcommands are resumable/idempotent, report exact input/output versions, and fail nonzero on unsupported or conflicting configuration. No paid side effects from benchmark/annotate without budget authorization. An existing launch flow may be called internally, not replaced by fake success.

24.2 Required completion artifacts

Produce environment.json, downloads.lock.json, asset-manifest.json, model-report.json, geometry-review.json, observation partitions/checksums, demand-profile manifest, schema/contract fixtures, annotation provenance, validation reports, benchmark logs, readiness.json, KNOWN_LIMITATIONS.md, RUNBOOK.md and ROLLBACK.md.

Reports distinguish fixtures / synthetic tests / user footage / independent references. Completion text lists files changed, commands run, result statuses, supported concurrency and all remaining gaps. A screenshot alone is not proof of correct tracking, conservation or forecasting.

25. Demo rehearsal and rollback

Rehearsal starts with source/mode disclosure and an actual playable clip. Show count windows and unavailable metrics, select a video-derived boundary profile, then demonstrate normal demand, a declared surge, C3 capacity reduction, emergency priority/recovery, a candidate comparison, operator approval at safe phase boundary, rejection/no-benefit case and manual takeover. Finish with audit and a reproducible replay.

Then deliberately stop an analytics camera, invalidate geometry, end a required input and cause an analysis timeout. Show explicit degraded/frozen states; no invented replacement flow. Emergency and incident demonstrations remain model events, not automatic detection claims from stock footage.

Rollback keeps raw media and original weights immutable, DB migrations backed up, source branch/tag and prior verified run package retained. Mode/provider changes create new runs. Stop owned workers, preserve audit/logs, and switch to the prior verified seeded/cached package with an explicit banner. Never run two authoritative engines on one run ID.

26. Commercial, scale and cost decisions

Current release is a demo, not a calibrated city deployment. Production needs licensed inputs/weights/runtime, authorized CCTV, camera/signal calibration, continuous failure handling, privacy/security review, service objectives, independent field/shadow evaluation and a separately approved actuation interface. An ITD dataset benchmark is not Bhubaneswar accuracy. [C01][E03][E04]

Cost controls: reuse downloaded assets and ITD; no default new dataset purchase, second coding-tool subscription, training run, cloud GPU or edge hardware. Track engineer/agent compute hours, actual hardware utilization, preview/storage size and downloaded bytes. Owner-configured paid-resource budget defaults to zero. No arbitrary percentage GPU/CPU saving is promised.

Architecture trade-off: one local inference owner plus aggregates minimizes duplication, but online throughput must be measured. Edge inference is a possible later deployment choice with device-management/updates/maintenance costs; centralized inference is another choice with bandwidth/failure-domain implications. Neither is mandated before a real deployment workload is specified.

27. Risk register and unresolved inputs

Risk / unknown	Agent handling	Release effect
Missing local model/video paths	Search only permitted roots; request exact mount/path once.	Data-dependent tasks blocked; code/fixture work continues.
ITD format/classes incompatible	Trusted isolated smoke test; preserve original; report exact error.	No silent detector substitution.
Unknown rights for intended demo	Record license and use purpose; seek proper authorization.	Rights badge/public packaging blocked, not fabricated.
Short/moving/occluded clips	Segment, reject, or visual-only role with evidence.	Reduced authoritative source coverage disclosed.
No independent annotation/count references	Agent-reviewed output and provisional tests.	No independent accuracy badge.
Twelve streams exceed hardware	Bounded processing, benchmark actual supported mode.	No realtime-12 claim; concurrent implementation still delivered.
Existing repo tests fail	Capture baseline and cause; do not erase unrelated user work.	Known limitations / repairs tracked separately.
Source-count/model mismatch	Preserve source semantics and declared virtual mapping.	No observed-real-corridor claim.
Forecast no better than persistence	Keep baseline, publish errors; do not force a challenger.	Forecast labeled baseline/unvalidated as appropriate.
Agent tool lacks persistent filesystem/GPU/browser	Detect capability; use supported local environment or report blocker.	No promise of execution the environment cannot provide.

Owner inputs that cannot be reliably inferred: authorized asset/model directories, intended permitted use and any account approvals, paid-resource ceiling and target presentation date. Hardware/filenames/package compatibility should be discovered by the agent where access permits rather than repeatedly asked as a substitute for inspection.

28. Requirement traceability and definition of done

Requirement ID	User need	Implemented by	Acceptance
REQ-01	Existing 12 videos	T01,T04	Twelve distinct assets accounted for; missing/duplicate/unsuitable disclosed.
REQ-02	Use ITD pretrained	T03,T06	Exact checkpoint and class map; no retraining dependency.
REQ-03	Proper detection/tracking/geometry	T05,T06,T14	Real ByteTrack, geometry evidence, crossing tests and review provenance.
REQ-04	Everything concurrently	T07,T16	Independent sessions/stages/forecasts; actual supported throughput report.
REQ-05	Lightweight intelligent forecasts	T08-T11	EWMA/persistence plus conserved rollout; Holt conditional.
REQ-06	No invented data	T02,T09,T11,T15	Null/status semantics, no leakage, exact accounting, factual templates.
REQ-07	Existing C1-C6 UI	T12,T13	Actual media + preserved graph + clear source/mode labels.
REQ-08	Agent does the work	T01-T17	Automated commands, draft geometry/labels, reports, blocked-action handling.
REQ-09	Know downloads/where/how	T03 + Section 8	Official sources, compatible locks, reuse first, rights/trust record.
REQ-10	Keep backend / no SUMO	T08,T12,T15	Go/Python roles preserved; clean default start without SUMO.
REQ-11	Signal comparison and safety	T11,T15	Equal-snapshot comparison; operator authority; safe virtual application.
REQ-12	Future production suitability	Section 26 + T17	Qualified roadmap, no unsupported production/accuracy/license claims.

Done means required code, real integration, tests, evidence and runbook are delivered with a truthful readiness matrix. It does not mean every target is green by assertion. A partially blocked package must say what works and what does not. No blanket "production ready", "hallucination free", "all automated and validated" or "12 cameras real time" without corresponding evidence.

Appendix A. Downloaded-video source register from the conversation

These eight source-page URLs were shortlisted in the conversation; they are provenance candidates, not independently revalidated downloads in this PRD. Agent must match actual downloaded files to source pages using file/source records, not invent an association from approximate visuals. No redownload is required when local files are present and permitted. The four earlier filenames are listed in Section 9; their original source URLs remain to be verified. [C01]

1. Mumbai intersection: [Pexels / video 33604950](#)
2. Indian intersection: [Pexels / video 30169851](#)
3. Crossroads clip: [Pixabay / crossroads 8698](#)
4. Mumbai traffic: [Pexels / video 30230719](#)
5. Urban overhead traffic: [Pexels / video 30608729](#)
6. Indian high-view street: [Pexels / video 30722590](#)
7. Bangalore overhead view: [Pexels / video 30144957](#)
8. Urban street/pedestrians: [Pexels / video 35022647](#)

Source license review entry points: www.pexels.com and [Pixabay / crossroads 8698](#). Agent verifies actual applicable rights and download terms at execution time; these links alone are not clearance for training, redistribution or every business use. No YouTube bypass/downloader is part of this build requirement because assets are already downloaded.

Appendix B. Source register

C01 is the conversation and user-pasted implementation discussions in this project. C02 is the uploaded `traffic_flow_aggregate_migration_report.md`, a static review at older commit

5ae1d5c914f8958721543c4c3b7bf8077f82d177. It supplies historical requirements and migration intent; current post-migration findings are tied to R sources, not attributed to the old report.

Repository R references are pinned to d6d774f6fb187ffc07925e0768add15ab53d548c, except R01's branch endpoint. Branch metadata was checked during PRD preparation. The baseline code findings were supplied/inspected in this conversation; no new runtime pass is claimed. E references are primary/official sources checked or linked with their verification status in `source_registry.json`. They explain tooling, not performance of this project.

[R01] Current main branch metadata

[Open source](#)

[R02] Existing agent architecture rules

[Open source](#)

[R03] Current sample vision pipeline

[Open source](#)

[R04] Aggregate runtime and demand source

[Open source](#) | [Related reference 1](#)

[R05] Finite-capacity flow kernel

[Open source](#)

[R06] Versioned aggregate metrics

[Open source](#)

[R07] Forecast rollouts and plan comparisons

[Open source](#)

[R08] Recent-flow demand estimator

[Open source](#)

[R09] Vision UI and fallback types

[Open source](#) | [Related reference 1](#)

[R10] Shared protobuf contracts

[Open source](#)

[R11] Shared Python dependency lock

[Open source](#)

[R12] C1-C6 network configuration

[Open source](#)

[R13] Intelligence tests[Open source](#)**[R14] Actual root build scripts/dependencies**[Open source](#)**[E01] Codex CLI / worktrees**[Open source](#) | [Related reference 1](#)**[E02] Google Antigravity download / project features**[Open source](#) | [Related reference 1](#)**[E03] ITD v1.2 model entry / model description**[Open source](#) | [Related reference 1](#)**[E04] ITD weights license and CC terms**[Open source](#) | [Related reference 1](#)**[E05] Python downloads**[Open source](#)**[E06] PyTorch local installation selector**[Open source](#)**[E07] Ultralytics installation / prediction**[Open source](#) | [Related reference 1](#)**[E08] Ultralytics tracking / explicit ByteTrack**[Open source](#)**[E09] FFmpeg downloads / ffprobe reference**[Open source](#) | [Related reference 1](#)**[E10] OpenCV Python headless package**[Open source](#)**[E11] NumPy installation**[Open source](#)**[E12] Node.js download**[Open source](#)**[E13] Go toolchains**[Open source](#)**[E14] Docker Compose installation**[Open source](#)

[E15] Ultralytics licensing[Open source](#)**[E16] CVAT auto-annotation SDK / installation**[Open source](#) | [Related reference 1](#)**[E17] Statsmodels damped Holt**[Open source](#)**[E18] ONNX Runtime installation**[Open source](#)**[E19] TensorRT installation**[Open source](#)**[E20] PyTorch checkpoint-loading warning**[Open source](#)**[E21] OpenCV geometric transforms**[Open source](#)

Official link provided; browser extraction failed. Reverify the API before implementation.

[E22] Daganzo: CTM network traffic[Open source](#) | [Related reference 1](#)**[E23] Exponentially weighted recurrence reference**[Open source](#)**[E24] Chronological time-series validation**[Open source](#)**[E25] Python concurrent futures / process workers**[Open source](#)**[E26] Protocol Buffers schema evolution**[Open source](#)

Appendix C. Installation and verification playbook

These instructions are for the execution agent. Resolve platform and permissions first. Do not treat illustrative shell commands as already executed, and do not ask the owner to manually perform each step when agent tools can do it.

C.1 Read-only preflight

Record current working directory, `git status --short`, `git rev-parse HEAD`, existing runtime versions and dependency manifests. Check `ffmpeg -version` and `ffprobe -version` when installed. For NVIDIA use an available approved GPU diagnostic; absent CUDA does not establish that all acceleration is unavailable on other hardware. Prefer capability probes over guessed device names.

Use the `ffprobe` pattern below after replacing the file with a discovered approved path. Quote paths with spaces. It is an existing tool command, unlike the proposed project CLI in Section 24. [E09]

```
ffprobe -v error -show_format -show_streams -of json "approved-clip.mp4"
```

Frame timestamps can be inspected with selected `-show_frames` fields; avoid materializing millions of frames in memory. Record timebase/rotation and decoder errors. Compute SHA-256 using the platform's approved utility or Python `hashlib`; no external upload is required.

C.2 Resolve isolated environments

Create core and vision environments using the discovered compatible Python interpreter (`python -m venv .venv`; `-vision` is the generic pattern). On Windows use the environment's `Scripts/python.exe`; on POSIX use `bin/python`. The interpreter actually selected and the command actually run must be recorded.

For the vision environment, select the PyTorch install command from the official OS/compute selector. Then resolve a compatible pinned Ultralytics version and required packages in a disposable environment. Record transitive dependencies, `pip check`, actual import smoke tests and package hashes. If Ultralytics brings `opencv-python`, do not additionally install a conflicting headless `cv2` distribution; choose one tested dependency strategy and document it. No hand-edited fake lock is acceptable. [E06][E07][E10]

Once a tested `services/vision/requirements.lock` exists, the clean-install acceptance pattern is:

```
<VISION_PYTHON> -m pip install -r services/vision/requirements.lock
<VISION_PYTHON> -m pip check
```

`<VISION_PYTHON>` is a configuration placeholder, not a literal executable. Use the package manager's hash-enforcement features where compatible with the resolved lock. Persist the exact PyTorch index/runtime origin separately if required. Verify installation on a clean environment before declaring reproducibility.

C.3 Preserve the existing application

Read actual root/workspace package scripts and Go manifests. Use existing `npm ci`, `build/typecheck/tests` and Go toolchain flows where present; do not assume old README framework names or an obsolete launcher filename. Start only the repository's intended PostgreSQL service; do not remove existing volumes. Record integration tests skipped because DB/private services are unavailable and later rerun them with isolated fixtures. [R14]

C.4 Optional annotation setup

Default is local annotation export. If CVAT is approved, use a pinned release from its official installation guidance in a separate development directory/Compose project and import labels through the SDK. CVAT may have its own infrastructure dependencies; these remain annotation-tool dependencies, not a reason to introduce Redis or extra services into the traffic application's runtime. Keep default development credentials private and never expose the tool publicly by default. [E16]

C.5 Offline-capable rehearsal

After permitted downloads, cache required wheels/assets locally where redistribution terms allow. Stop external network use for the numerical demo and confirm cached and seeded modes still operate. Build-time agents may require network access; this does not make runtime LLM calls permissible. Package source/metadata and reconstruction instructions rather than redistributing model/video binaries without rights.

Appendix D. Delivery notes and change control

v1.0.1 records the owner's confirmation that ITD v1.2 is already downloaded. The existing checkpoint is the required first input; no default redownload. Local path, file integrity, class mapping and runtime compatibility still require agent verification. Agent kickoff, task plan and configuration template are aligned with this asset status. All other requirements are unchanged.

v1.0 consolidates the conversation, explicitly adds dual processing modes, separates agent execution from independent validation, and adds causal delayed-bin supply, readiness labels, download trust controls and acceptance targets. Those are proposed design resolutions, not previously measured features.

Before implementation, the lead agent must produce a brief delta against execution HEAD and record any real incompatibilities. Changes to topology, model substitution, source semantics, safety authority, paid-resource use, independent-accuracy claims or acceptance thresholds require explicit versioned review. No source narrative should be silently rewritten to make the implementation look complete.

Final product statement: Video-derived, aggregate traffic decision-support demonstration. Actual sample-video analytics, modeled connected network, causal lightweight forecasts and operator-approved virtual timing. No claim of synchronized real-city observations, independently validated field performance or physical signal control.